

Reviewer Pack — Minimal Rank of Eilenberg-MacLane Spaces Bounds Polynomial H...

Entry 9b2ebf8bb8d1 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 18:23:44 UTC

Minimal Rank of Eilenberg-MacLane Spaces Bounds Polynomial Hierarchy Depth

Entry ID: 9b2ebf8bb8d1 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 18:23:44 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Eilenberg-MacLane Spaces) **Field B** (complexity object): Complexity Theory: Polynomial Hierarchy Complexity

Statement:

[‘For each fixed integer $k \geq 2$, there exists a constant $c_k > 0$ such that for every k -CNF formula with n variables, the minimal rank of its associated Eilenberg-MacLane space $A_k(n)$ is upper-bounded by cn^{k-1} . Equivalently, if there exists a k -CNF formula F with n variables such that $\text{rank}(A_k(n)) > cn^{k-1}$, then F does not belong to PH.’, ‘For every language $L \in P^k$, the minimal rank of its associated Eilenberg-MacLane space $A_L(n)$ is at least cn^{k-1} for some constant c .’]

Rationale (proposer’s reasoning):

[‘The use of algebraic topology in complexity theory has been limited to topological invariants like homology and cohomology, which are not directly applicable to the polynomial hierarchy. Eilenberg-MacLane spaces provide a different class of topological spaces that can potentially capture the complexity of functions and languages.’, ‘Eilenberg-MacLane spaces have been used in algebraic topology to study group structures, and their ranks can be computationally determined using algorithms like the cellular approximation theorem. This suggests that they could serve as a novel tool for analyzing complexity classes.’]

Taxonomy category: Algebraic_Topology (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 76d2be81924e9ae3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if no seed generates a k -CNF formula with $\text{rank}(A_k(n)) > cn^{k-1}$ for any $n \leq 40$ and $k \geq 2$, where $\text{support_fraction} \geq 0.9$ AND $\text{metric_mean} \leq 3$, across all seeds.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - Minimal Rank Eilenberg-MacLane Spaces AND Polynomial Hierarchy - CNF formula AND Eilenberg-MacLane space rank AND complexity theory - Polynomial Hierarchy depth AND upper bound ON Eilenberg-MacLane space rank

Top relevant hits considered: - [<http://arxiv.org/abs/2206.08165v3>] The $H\mathbb{F}_2$ -homology of C_2 -equivariant Eilenberg-MacLane spaces - [<http://arxiv.org/abs/1201.6222v3>] Polynomial-time homology for simplicial Eilenberg-MacLane spaces - [<http://arxiv.org/abs/1111.7220v1>] Algebraic extensions of an Eilenberg-MacLane spectrum - [<http://arxiv.org/abs/1102.2932v2>] Applications of Monotone Rank to Complexity Theory - [<http://arxiv.org/abs/1612.03553v5>] Theory of Fundamental Bessel Functions of High Rank - [<http://arxiv.org/abs/1710.06378v1>] The Hard Problems Are Almost Everywhere For Random CNF-XOR Formulas - [<http://arxiv.org/abs/1710.06598v2>] A Bounded Degree Lasserre Hierarchy with SOCP Relaxations for Global Polynomial Optimization and Applications - [<http://arxiv.org/abs/1104.2412v2>] Stanley depth and complete k -partite hypergraphs

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(k, n):
        cnf = []
        for _ in range(n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(k)]
            cnf.append(clause)
        return cnf

    def rank_eilenberg_mac_lane_space(cnf_formula):
        # Placeholder function to compute the rank of  $A_k(n)$ 
        # This is a dummy implementation and should be replaced with actual computation
        return len(cnf_formula) # Simplified for demonstration purposes
```

```

k_values = [2, 3, 4, 5]
n = 10
total_rank = 0
instances_tested = 0

for k in k_values:
    cnf_formula = generate_k_cnf(k, n)
    rank = rank_eilenberg_mac_lane_space(cnf_formula)
    total_rank += rank
    instances_tested += 1

metric_value = total_rank / len(k_values)
conjecture_holds = True
counterexample = ""

if metric_value > 3:
    conjecture_holds = False
    counterexample = "metric_value_exceeds_threshold"

return {
    "metric_name": "Average Rank",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    metric_values = [r["metric_value"] for r in results]
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        result = "SUPPORTED"
    elif support_fraction >= 0.9 and max(metric_values) <= 3:
        result = "SUPPORTED"
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        result = f"FALSIFIED counterexample=\"metric_value_exceeds_threshold\" first_failing_seed={first_failing_seed}"

    print(f"RESULT: {result} mean={sum(metric_values) / len(metric_values):.2f} std={math.sqrt(sum((m - sum(metric_values) / len(metric_values)) ** 2 for m in metric_values)) / len(metric_values):.2f}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ue': 10.0, 'instances_tested': 4, 'conjecture_holds': False, 'counterexample': 'metric_value_exceeds
TRIAL: {'metric_name': 'Average Rank', 'metric_value': 10.0, 'instances_tested': 4, 'conjecture_hold
TRIAL: {'metric_name': 'Average Rank', 'metric_value': 10.0, 'instances_tested': 4, 'conjecture_hold
TRIAL: {'metric_name': 'Average Rank', 'metric_value': 10.0, 'instances_tested': 4, 'conjecture_hold
TRIAL: {'metric_name': 'Average Rank', 'metric_value': 10.0, 'instances_tested': 4, 'conjecture_hold
TRIAL: {'metric_name': 'Average Rank', 'metric_value': 10.0, 'instances_tested': 4, 'conjecture_hold
TRIAL: {'metric_name': 'Average Rank', 'metric_value': 10.0, 'instances_tested': 4, 'conjecture_hold
TRIAL: {'metric_name': 'Average Rank', 'metric_value': 10.0, 'instances_tested': 4, 'conjecture_hold
TRIAL: {'metric_name': 'Average Rank', 'metric_value': 10.0, 'instances_tested': 4, 'conjecture_hold
```

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a86a8d40.py", line 78, in <module>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a86a8d40.py", line 78, in <genexpr>
    first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
                        ~~~~~^
```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to verify the conjecture. | next: Re-run the test code ensuring that it completes without crashing and produces results for further analysis.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	12597
2	preregistration	ollama_remote	glm4:latest	0	0	5648
3	novelty	ollama_remote	glm4:latest	0	0	4713
4	novelty	ollama_remote	glm4:latest	0	0	7693
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12353
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7524
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8383
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7896
9	judge	ollama_remote	glm4:latest	0	0	8566

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 75374 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/9b2ebf8bb8d1.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/9b2ebf8bb8d1.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/9b2ebf8bb8d1.tar.gz` (if generated)